

Embodied Intelligence Club – Christmas Project

Multi-Agent Mobile Robot Soccer (Wheeled Robots)

Konstantinos Chatzilygeroudis

December 23, 2025

Language: Python or C++

Work style: Team or individual

Goal

Build a small **multi-agent soccer simulator** for **differential-drive robots**, emphasizing:

- **explicit use of classic design patterns:** Strategy, Factory Method, Template Method, Observer
- **linear algebra** via graph Laplacians (formation control / consensus), passing geometry, and least squares.

Mandatory Design Patterns

Your code must use these patterns in a visible, defensible way:

1. **Strategy:** interchangeable dynamics models and team/robot policies.
2. **Factory Method:** create dynamics/policies/scenarios from config (no direct constructors outside factories).
3. **Template Method:** fixed `step()` skeleton for the simulator.
4. **Observer:** scoreboard/metrics/plots implemented as observers; the simulator core never prints/plots.

What you will build

A 2D soccer world with:

- N robots per team (recommend $N = 3$ for baseline),
- a ball with simple 2D motion + friction,
- policies for formation keeping, role assignment (striker/defender/support), passing/shooting, interception,
- observers computing match metrics (score, possession, formation error, interceptions).

Dynamics: Differential-drive kinematics

Each robot has state $x = [p_x, p_y, \theta]^T$ and wheel commands (v_L, v_R) . Let wheel radius be r and wheel-base (distance between wheels) be b . Define body linear and angular velocity:

$$v = \frac{r}{2}(v_R + v_L), \quad \omega = \frac{r}{b}(v_R - v_L).$$

Discrete-time update with time-step Δt :

$$\begin{aligned} p_{x,k+1} &= p_{x,k} + \Delta t v_k \cos \theta_k, \\ p_{y,k+1} &= p_{y,k} + \Delta t v_k \sin \theta_k, \\ \theta_{k+1} &= \theta_k + \Delta t \omega_k. \end{aligned}$$

Note: If you prefer, your policy may compute (v, ω) directly, and the dynamics strategy converts it to (v_L, v_R) , or vice versa—but the differential-drive model must appear clearly in the code.

Ball motion

Model the ball as a point mass with velocity decay ($b_k \in \mathbb{R}^2, u_k^{\text{kick}} \in \mathbb{R}^2$):

$$\begin{aligned} b_{k+1} &= b_k + \Delta t \dot{b}_k, \\ \dot{b}_{k+1} &= \alpha \dot{b}_k + u_k^{\text{kick}}, \end{aligned}$$

with $\alpha \in (0, 1)$. When a robot kicks, apply an impulse u_k^{kick} in a chosen direction.

Core linear algebra tasks

Your submission must include all of the following:

1. a non-trivial matrix operation with shape asserts,
2. at least one **linear system solve** using `solve` or least squares (not explicit inverse),
3. a **graph Laplacian / consensus formation control** component (explained below).

Graph Laplacian and Formation Consensus

Communication graph

For one team with N robots, define an undirected communication graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where:

- $\mathcal{V} = \{1, 2, \dots, N\}$ is the set of robots,
- $(i, j) \in \mathcal{E}$ means robots i and j exchange information (or “try to maintain” a relative formation constraint).

For baseline, use a simple graph such as a chain, ring, or fully connected graph.

Adjacency matrix A

Define the adjacency matrix $A \in \mathbb{R}^{N \times N}$ as:

$$A_{ij} = \begin{cases} 1 & \text{if } (i, j) \in \mathcal{E}, \\ 0 & \text{otherwise,} \end{cases} \quad A_{ii} = 0.$$

For undirected graphs: $A = A^T$.

Degree matrix D

Define the degree of node i as:

$$d_i = \sum_{j=1}^N A_{ij}.$$

Then the degree matrix $D \in \mathbb{R}^{N \times N}$ is diagonal:

$$D = \text{diag}(d_1, d_2, \dots, d_N).$$

Graph Laplacian L

The (combinatorial) graph Laplacian is:

$$L = D - A.$$

Key properties you should mention in your report:

- L is symmetric positive semi-definite for undirected graphs.
- $L\mathbf{1} = 0$, so $\mathbf{1}$ is always an eigenvector with eigenvalue 0.
- The number of zero eigenvalues equals the number of connected components; if the graph is connected, the smallest eigenvalue is 0 and the second-smallest (*algebraic connectivity*) is > 0 .

Why L creates consensus

Let $q_i \in \mathbb{R}^m$ be a quantity each robot tries to agree on (e.g., position offset error). A classic consensus dynamics is:

$$\dot{q}_i = - \sum_{j=1}^N A_{ij} (q_i - q_j).$$

Stack $q = [q_1^T, \dots, q_N^T]^T \in \mathbb{R}^{mN}$. Then:

$$\dot{q} = -(L \otimes I_m)q,$$

where $\otimes$ is the Kronecker product and I_m the $m \times m$ identity. If the graph is connected, this drives all q_i to the same value (consensus).

Formation tracking with Laplacian

Let $p_i \in \mathbb{R}^2$ be the position of robot i . Define stacked team position:

$$P = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_N \end{bmatrix} \in \mathbb{R}^{2N}.$$

Let P^* represent a desired formation shape (e.g., triangle) in the field frame.

One simple formation controller is:

$$\dot{P} = -k(L \otimes I_2)(P - P^*),$$

with gain $k > 0$. Discretize:

$$P_{k+1} = P_k + \Delta t \dot{P}_k.$$

Interpretation: each robot moves based on differences between its own formation error and its neighbors' formation errors.

Important: the positioning information that each robot has should not be global (i.e. the robots do not know their global position, just relative to their neighbors).

From desired velocity to differential-drive commands

The consensus law gives a desired planar velocity for robot i :

$$v_i^{\text{des}} = \dot{p}_i \in \mathbb{R}^2.$$

Convert this to differential-drive controls:

- Desired heading: $\theta_i^{\text{des}} = \text{atan2}(v_{i,y}^{\text{des}}, v_{i,x}^{\text{des}})$.
- Heading error: $e_\theta = \text{wrap}(\theta_i^{\text{des}} - \theta_i)$.
- Choose linear speed $v = \|v_i^{\text{des}}\|$ (clamped), angular speed $\omega = k_\theta e_\theta$.
- Convert (v, ω) to wheel speeds:

$$v_R = \frac{1}{r} \left(v + \frac{b}{2} \omega \right), \quad v_L = \frac{1}{r} \left(v - \frac{b}{2} \omega \right).$$

Passing / interception

Passing direction geometry (example)

Let the ball be at b , teammate t , and goal center g . Define unit vectors:

$$d_t = \frac{t - b}{\|t - b\|}, \quad d_g = \frac{g - b}{\|g - b\|}.$$

A simple pass score:

$$\text{score}(t) = w_g d_t^T d_g - w_o \sum_j \phi(d_t^T d_{o_j}),$$

where $d_{o_j} = \frac{o_j - b}{\|o_j - b\|}$ points to opponent j , and $\phi(\cdot)$ penalizes alignment with blockers (you define ϕ). You can use this to compute which teammate is best to pass the ball to.

Interception (example)

Predict a short-horizon ball position:

$$b_{\text{pred}} = b + \tau \dot{b}.$$

Pick τ by scanning a small set (e.g., $\tau \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ seconds) and choose the earliest reachable intercept point given max speed.

Core software architecture

Strategy Interfaces

- `DynamicsModel`:
 - `propagate(state, action, dt) -> next_state`
 - `state.dim() -> int`
- `Policy` (robot-level or team-level):
 - `compute_actions(world) -> actions`

Factory Method

All creation must happen through factories driven by config:

- `DynamicsFactory.create(cfg) -> DynamicsModel`
- `PolicyFactory.create(cfg) -> Policy`
- `ScenarioFactory.create(cfg) -> SoccerScenario`

Rule: No direct instantiation of concrete classes outside factory code.

Template Method

A base `SoccerSimulator` provides `step()`:

1. compute actions from each team policy (or per-robot policies)
2. propagate robot states with differential-drive dynamics
3. update ball state (including kicks/collisions)
4. enforce world rules (bounds, goals, resets)
5. notify observers

Observer Pattern

Define an `Observer` interface:

`update(event_dict).`

Provide `attach/detach/notify`. Observers must do all logging, plotting, and metrics.

Required implementations

You must implement at least:

- **2** dynamics strategies (choose 2):
 - `DifferentialDriveDynamics` (required baseline)
 - `DifferentialDriveWithActuationNoise`
 - `DifferentialDriveWithSlip` (optional realism)
- **2** policy strategies (choose 2):
 - `GoToPointPolicy` (baseline navigation)
 - `FormationConsensusPolicy` (uses L and $L \otimes I_2$)
 - `SoccerHeuristicPolicy` (roles: striker/defender/support)
 - `PassOrShootPolicy` (pass selection using dot products / LS)
- **2** observers:
 - `ScoreboardObserver` (score, shots)
 - plus one: `PossessionObserver`, `InterceptionObserver`, or `TeamShapeObserver`

Scenario rules

Implement:

- Field boundaries: reflect, clamp, or penalize leaving the field.
- Goals: define goal regions; increment score; reset ball/robots on goal.
- Kicking: when a robot is within radius r_k of the ball, it may apply a kick impulse.
- Match length: fixed number of steps or simulated time.

Deliverables

1. **Code repository** with:
 - `src/` (core), `configs/` (configs), `demo/` (runs)
 - separation for patterns: `strategies/`, `factories/`, `observers/`, `sim/`
2. **Pattern map (1 page)**: Pattern $\rightarrow$ classes $\rightarrow$ justification.
3. **Demo** that runs at least 3 configs by changing config only.
4. **Plots/visuals**:
 - 2D trajectories (teams + ball)
 - score over time (or final score + possession fraction)
 - formation error $\|P - P^*\|$

Notes / constraints

- Keep it minimal: correctness and architecture over fancy graphics.
- You may use `numpy` and `matplotlib`.
- Avoid using full soccer simulators—implement the minimal world yourself.